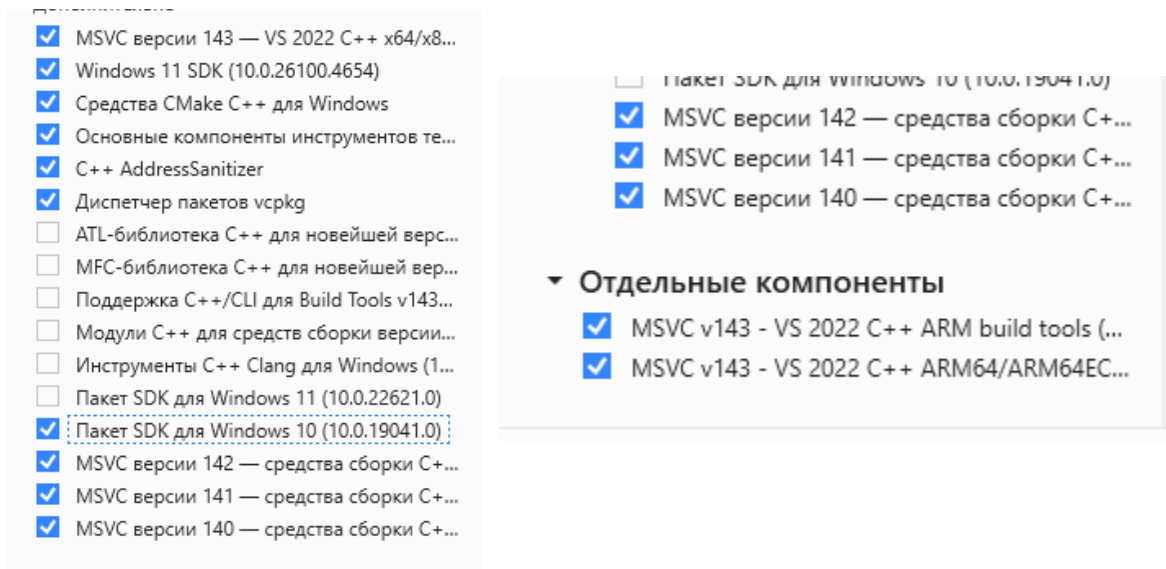


Аналіз машинного коду для x86/x64 та ARM/ARM64

Кувавина Софія

Мета роботи: Отримати навички розпізнавання констукцій мов високого рівня в машинному коді для архітектур x86/x64 та ARM/ARM64 на прикладі C/C++.

Для коректної роботи я встановлюю додатково для наших архітектур



1.1 Проаналізувати машинний код прикладу hanoi.c для Windows x64, ARM, ARM64 (MSVC), для Linux amd64, arm, arm64 (GCC), для Linux amd64 (LLVM clang, <https://llvm.org/>);

Windows

Скомпілювали для amd64, x86_arm, x86_arm64

x86_arm64

```

C:\lab1\hanoi>cd /d "C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build"
C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build>vcvarsall.bat x86_arm64
*****
** Visual Studio 2022 Developer Command Prompt v17.14.13
** Copyright (c) 2025 Microsoft Corporation
*****
[vcvarsall.bat] Environment initialized for: 'x86_arm64'

C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build>cd /d C:\lab1\hanoi
C:\lab1\hanoi>cl /FAcs hanoi.c /Fehanoi_arm64.exe /Fahanoi_arm64.cod
Оптимизирующий компилятор Microsoft (R) C/C++ версии 19.38.33145 для ARM64
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

hanoi.c
Microsoft (R) Incremental Linker Version 14.38.33145.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:hanoi_arm64.exe
hanoi.obj
C:\lab1\hanoi>_

```

Amd64

```

C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build>vcvarsall.bat x64
*****
** Visual Studio 2022 Developer Command Prompt v17.14.13
** Copyright (c) 2025 Microsoft Corporation
*****
[vcvarsall.bat] Environment initialized for: 'x64'

C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build>cd /d C:\lab1\hanoi
C:\lab1\hanoi>cl /FAcs hanoi.c /Fehanoi_x64.exe /Fahanoi_x64.cod
Оптимизирующий компилятор Microsoft (R) C/C++ версии 19.44.35215 для x64
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

hanoi.c
Microsoft (R) Incremental Linker Version 14.44.35215.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:hanoi_x64.exe
hanoi.obj
C:\lab1\hanoi>

```

І аналогічно для **x86_arm**

```

C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools>cd /d C:\lab1\hanoi
C:\lab1\hanoi>cl hanoi.c /FAcs /Fehanoi_x86.exe /Fahanoi_x86.cod
Оптимизирующий компилятор Microsoft (R) C/C++ версии 19.38.33145 для x86
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

hanoi.c
Microsoft (R) Incremental Linker Version 14.38.33145.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:hanoi_x86.exe
hanoi.obj

```

Отримали наступні файли

hanoi.c	04.09.2025 23:58	C Source	1 КБ
hanoi.obj	06.09.2025 19:16	3D Object	2 КБ
hanoi_arm.cod	05.09.2025 22:01	Файл "COD"	9 КБ
hanoi_arm64.cod	05.09.2025 21:35	Файл "COD"	8 КБ
hanoi_arm64.exe	05.09.2025 21:35	Приложение	138 КБ
hanoi_x64.cod	05.09.2025 22:10	Файл "COD"	7 КБ
hanoi_x64.exe	05.09.2025 22:10	Приложение	114 КБ
hanoi_x86.cod	06.09.2025 19:16	Файл "COD"	7 КБ
hanoi_x86.exe	06.09.2025 19:16	Приложение	114 КБ

Тепер подивимося відмінності між різними архітектурами

```
hanoi_arm64.cod – Блокнот
Файл  Правка  Формат  Вид  Справка

EXPORT  |__local_stdio_printf_options|
EXPORT  |_vfprintf_l|
EXPORT  |printf|
EXPORT  |hanoi|
EXPORT  |main|
IMPORT  |__acrt_iob_func|
IMPORT  |__stdio_common_vfprintf|
```

```
hanoi_x86.cod – Блокнот
Файл  Правка  Формат  Вид  Справка

PUBLIC  __local_stdio_printf_options
PUBLIC  __vfprintf_l
PUBLIC  _printf
PUBLIC  _hanoi
PUBLIC  _main
EXTRN   __acrt_iob_func:PROC
```

У x86 використовується директива PUBLIC, яка перелічує функції, доступні для використання іншими модулями, у ARM64 замість цього застосовується EXPORT

```

hanoi_x64.cod - Блокнот
Файл  Правка  Формат  Вид  Справка

00003 6a 42      push    66          ; 00000042H
00005 6a 43      push    67          ; 00000043H
00007 6a 41      push    65          ; 00000041H
00009 6a 03      push    3
0000b e8 00 00 00 00 call    _hanoi
00010 83 c4 10      add     esp, 16          ; 000\

```

```

hanoi_x86.cod - Блокнот
Файл  Правка  Формат  Вид  Справка

00003 6a 42      push    66
00005 6a 43      push    67
00007 6a 41      push    65
00009 6a 03      push    3
0000b e8 00 00 00 00 call    _hanoi
00010 83 c4 10      add     esp, 16

```

```

hanoi_arm64.cod - Блокнот
Файл  Правка  Формат  Вид  Справка

; 14 :      hanoi(3, 'A', 'C', 'B');

00008 52800843      mov     w3,#0x42
0000c 52800862      mov     w2,#0x43
00010 52800821      mov     w1,#0x41
00014 52800060      mov     w0,#3
00018 94000000      bl      hanoi

```

У amd64, x86 виклик функцій здійснюється інструкцією call, тоді як у ARM та ARM64 для цього використовується інструкція bl (branch with link)

```

hanoi_x64.cod - Блокнот
Файл  Правка  Формат  Вид  Справка

; 7 :      hanoi(n-1, from, aux, to);

00022 0f b6 55 10      movzx   edx, BYTE PTR _to$[ebp]
00026 52      push    edx
00027 0f b6 45 14      movzx   eax, BYTE PTR _aux$[ebp]
0002b 50      push    eax
0002c 0f b6 4d 0c      movzx   ecx, BYTE PTR _from$[ebp]
00030 51      push    ecx
00031 8b 55 08      mov     edx, DWORD PTR _n$[ebp]
00034 83 ea 01      sub     edx, 1
00037 52      push    edx
00038 e8 00 00 00 00      call    _hanoi
0003d 83 c4 10      add     esp, 16          ; 00000010H

```

```

hanoi_x86.cod - Блокнот
Файл  Правка  Формат  Вид  Справка

00009 0f be 45 10      movsx   eax, BYTE PTR _to$[ebp]
0000d 50      push    eax
0000e 0f be 4d 0c      movsx   ecx, BYTE PTR _from$[ebp]
00012 51      push    ecx
00013 68 00 00 00 00      push    OFFSET $SG9672
00018 e8 00 00 00 00      call    _printf
0001d 83 c4 0c      add     esp, 12
00020 eb 57      jmp     SHORT $LN1@hanoi
$LN2@hanoi:

```

```
hanoi_arm64.cod - Блокнот
Файл  Правка  Формат  Вид  Справка
; 5      :      printf("Move disk 1 from %c to %c\n", from, to);

00030 39c047e2      ldrsb    w2,[sp,#0x11]
00034 2a0203e2      mov     w2,w2
00038 39c043e1      ldrsb    w1,[sp,#0x10]
0003c 2a0103e1      mov     w1,w1
00040 2a0203e2      mov     w2,w2
00044 2a0103e1      mov     w1,w1
00048 90000008      adrp    x8,|$SG4985|
0004c 91000100      add     x0,x8,|$SG4985|
00050 94000000      bl      printf
00054 14000019      b       |$LN3@hanoi|
00058      |$LN2@hanoi|
```

```
hanoi_arm.cod - Блокнот
Файл  Правка  Формат  Вид  Справка

0000e f99d 2010      ldrsb    r2,[sp,#0x10]
00012 f99d 100c      ldrsb    r1,[sp,#0xC]
00016 4813          ldr      r0,|$LN8@hanoi|    ; =|$SG5062|
00018 f000 f800      bl      printf
0001c e01b          b       |$LN3@hanoi|
0001e      |$LN2@hanoi|
```

ARM та ARM64 використовується інструкція `ldrsb`, яка завантажує байт із пам'яті та розширює його зі знаком

У amd64 x86 для цього застосовується інструкція `movzx`, яка розширює байт без врахування знака

```
hanoi_arm.cod - Блокнот
Файл  Правка  Формат  Вид  Справка

; 90      :      {
00000 e92d 4800      push    r11,lr}
00004 46eb          mov     r11,sp
00006 b082          sub     sp,sp,#8
00008      |$M6|

; 91      :      static unsigned __int64 _OptionsStorage;
; 92      :      return &_OptionsStorage;
```

```
hanoi_x64.cod - Блокнот
Файл  Правка  Формат  Вид  Справка
; 3      :      void hanoi(int n, char from, char to, char aux) {

00000 55          push    ebp
00001 8b ec          mov     ebp, esp

; 4      :      if (n == 1)

00003 83 7d 08 01      cmp     DWORD PTR _n$[ebp], 1
00007 75 19          jne     SHORT $LN2@hanoi
```

```
hanoi_x86.cod - Блокнот
Файл  Правка  Формат  Вид  Справка

00000 55          push    ebp
00001 8b ec          mov     ebp, esp
00003 83 ec 08          sub     esp, 8
```

```
hanoi_arm64.cod - Блокнот
Файл  Правка  Формат  Вид  Справка
; 644 : {
00000 |$LN3|
00000 a9bd7bfd stp fp,lr,[sp,#-0x30]!
00004 910003fd mov fp,sp
00008 f90017e0 str x0,[sp,#0x28]
0000c f90013e1 str x1,[sp,#0x20]
00010 f9000fe2 str x2,[sp,#0x18]
00014 f9000be3 str x3,[sp,#0x10]
```

у amd64, x86 зберігається попереднє значення евр,
у ARM реєстри r11 і lr,
а у ARM64 реєстри fp та lr за допомогою інструкції stp

Linux

Встановлюємо потрібні нам пакети для kali

```
sudo apt install gcc gcc-i686-linux-gnu gcc-arm-linux-gnueabi gcc-aarch64-linux-gnu
binutils binutils-arm-linux-gnueabi binutils-aarch64-linux-gnu
```

```
(kali@kali)-[~/lab1/hanoi]
$ which gcc
gcc --version
which objdump
objdump --version

/usr/bin/gcc
gcc (Debian 14.2.0-19) 14.2.0
Copyright (C) 2024 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

/usr/bin/objdump
GNU objdump (GNU Binutils for Debian) 2.45
Copyright (C) 2025 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or (at your option) any later version.
This program has absolutely no warranty.
```

Генерація асемблерних листингів (GCC)

Далі виконую команди відповідно до методички для створення лістингів

AMD64

```
(kali@kali)-[~/lab1/hanoi]
$ gcc -g hanoi.c -o hanoi.amd64

(kali@kali)-[~/lab1/hanoi]
$ ./hanoi.amd64
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```



```

(kali㉿kali)-[~/lab1/hanoi]
$ gcc -Wa,-adhln -g hanoi.c > hanoi.amd64.lst

(kali㉿kali)-[~/lab1/hanoi]
$ less hanoi.amd64.lst

(kali㉿kali)-[~/lab1/hanoi]
$ objdump -drwC -Mintel -S hanoi.amd64 > hanoi.amd64.objdump.lst

```

ARM

```

(kali㉿kali)-[~/lab1/hanoi]
$ arm-linux-gnueabi-gcc -O0 -g -o hanoi.arm hanoi.c

(kali㉿kali)-[~/lab1/hanoi]
$ arm-linux-gnueabi-gcc -O0 -g -c -Wa,-adhln hanoi.c -o hanoi.arm.o > hanoi.arm.lst

(kali㉿kali)-[~/lab1/hanoi]
$ arm-linux-gnueabi-objdump -d -C -S hanoi.arm > hanoi.arm.objdump.lst

```

```

52 0030 0B304BE5  stlb    r3, [fp, #-11]
4:hanoi.c      ****    if (n == 1)
53             .loc 1 4 8
54 0034 08301BE5    ldr     r3, [fp, #-8]
55 0038 010053E3    cmp     r3, #1
56 003c 0700001A    bne     .L2
5:hanoi.c      ****    printf("Move disk 1 from %c to %c\n", from, to);
57             .loc 1 5 9
58 0040 09305BE5    ldrb    r3, [fp, #-9] @ zero_extendqisi2
59 0044 0A205BE5    ldrb    r2, [fp, #-10] @ zero_extendqisi2
60 0048 0310A0E1    mov     r1, r3
61 004c 60309FE5    ldr     r3, .L5
62             .LPIC0:
63 0050 03308FE0    add     r3, pc, r3
64 0054 0300A0E1    mov     r0, r3
65 0058 FFFFFFFB    bl      printf(PLT)
6:hanoi.c      ****    else {
7:hanoi.c      ****    hanoi(n-1, from, aux, to);
8:hanoi.c      ****    printf("Move disk %d from %c to %c\n", n, from, to);
9:hanoi.c      ****    hanoi(n-1, aux, to, from);
10:hanoi.c     ****    }
11:hanoi.c     **** }

```

ARM64

```
(kali@kali)-[~/lab1/hanoi]
$ aarch64-linux-gnu-gcc -O0 -g -o hanoi.arm64 hanoi.c

(kali@kali)-[~/lab1/hanoi]
$ aarch64-linux-gnu-gcc -O0 -g -c -Wa,-adhln hanoi.c -o hanoi.arm64.o > hanoi.arm64.lst

(kali@kali)-[~/lab1/hanoi]
$ aarch64-linux-gnu-objdump -d -C -S hanoi.arm64 > hanoi.arm64.objdump.lst
```

```
4:hanoi.c      ****      if (n == 1)
31             .loc 1 4 8
32 0018 E01F40B9      ldr     w0, [sp, 28]
33 001c 1F040071      cmp     w0, 1
34 0020 21010054      bne     .L2
5:hanoi.c      ****      printf("Move disk 1 from %c to %c\n", from, to);
35             .loc 1 5 9
36 0024 E06F4039      ldrb    w0, [sp, 27]
37 0028 E16B4039      ldrb    w1, [sp, 26]
38 002c E203012A      mov     w2, w1
39 0030 E103002A      mov     w1, w0
40 0034 00000090      adrp    x0, .LC0
41 0038 00000091      add     x0, x0, :lo12:LC0
42 003c 00000094      bl      printf
6:hanoi.c      ****      else {
7:hanoi.c      ****      hanoi(n-1, from, aux, to);
8:hanoi.c      ****      printf("Move disk %d from %c to %c\n", n, from, to);
9:hanoi.c      ****      hanoi(n-1, aux, to, from);
10:hanoi.c     ****      }
11:hanoi.c     ****      }
43             .loc 1 11 1
44 0040 15000014      b       .L4
45             .L2:
7:hanoi.c      ****      printf("Move disk %d from %c to %c\n", n, from, to);
46             .loc 1 7 9
47 0044 E01F40B9      ldr     w0, [sp, 28]
48 0048 00040051      sub     w0, w0, #1
49 004c E36B4039      ldrb    w3, [sp, 26]
50 0050 E2674039      ldrb    w2, [sp, 25]
51 0054 E16F4039      ldrb    w1, [sp, 27]
52 0058 00000094      bl      hanoi
8:hanoi.c      ****      hanoi(n-1, aux, to, from);
53             .loc 1 8 9
54 005c E06F4039      ldrb    w0, [sp, 27]
55 0060 E16B4039      ldrb    w1, [sp, 26]
56 0064 E303012A      mov     w3, w1
57 0068 E203002A      mov     w2, w0
```

Відмінності

1. amd64, arm64

Рядкові літерали визначаються за допомогою директиви `.string`

```
8             .LC0:
9 0000 4D6F7665      .string "Move disk 1 from %c to %c\n"
9             20646973
9             68203120
9             66726F60
9             20256320
10 001b 00000000      .align 3
10             00
11             .LC1:
12 0020 4D6F7665      .string "Move disk %d from %c to %c\n"
12             20646973
13             68203120
```

Arm

Рядки зберігаються за допомогою директиви `.ascii` і додають спеціальні символи

Тут видно \012\000 - це символ нового рядка LF (\n) + NULL-термінатор

```
18      .align 2
19      .LC0:
20 0000 4D6F7665      .ascii "Move disk 1 from %c to %c\012\000"
20      20646973
20      6B203120
20      66726F6D
20      20256320
21 001b 00      .align 2
22      .LC1:
23 001c 4D6F7665      .ascii "Move disk %d from %c to %c\012\000"
23      20646973
```

2. AMD64

Використовується система передачі аргументів SysV ABI:

параметри приходять у регістрах %edi, %esi, %edx, %ecx, а потім зберігаються у стеку

```
pushq   %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq    %rsp, %rbp
.cfi_def_cfa_register 6
subq    $16, %rsp
movl    %edi, -4(%rbp)
movl    %ecx, %eax
movl    %esi, %ecx
movb    %cl, -8(%rbp)
movb    %dl, -12(%rbp)
movb    %al, -16(%rbp)
```

ARM

У ARM усі чотири параметри приходять у r0, r1, r2, r3

Вони явно перекладаються у стек

```
push    {fp, lr}
.cfi_def_cfa_offset 8
.cfi_offset 11, -8
.cfi_offset 14, -4
add     fp, sp, #4
.cfi_def_cfa 11, 4
sub     sp, sp, #8
str     r0, [fp, #-8]
mov     r0, r1
mov     r1, r2
mov     r2, r3
mov     r3, r0
strb    r3, [fp, #-9]
mov     r3, r1
strb    r3, [fp, #-10]
mov     r3, r2
strb    r3, [fp, #-11]
(n = 1)
```

ARM64

Використовуються 64-бітні регістри x29 (frame pointer) та x30 (link register)

Параметри приходять у x0...x3, але для збереження символів беруться тільки молодші частини w0...w3

```

stp    x29, x30, [sp, -32]!
.cfi_def_cfa_offset 32
.cfi_offset 29, -32
.cfi_offset 30, -24
mov     x29, sp
str     w0, [sp, 28]
strb    w1, [sp, 27]
strb    w2, [sp, 26]
strb    w3, [sp, 25]

```

3. AMD64

Аргументи передаються за SysV ABI:

1-й - %rdi, 2-й - %rsi, 3-й - %rdx.

Для отримання адреси рядка використовується leaq

```

.LC0:
movsbl  -12(%rbp), %edx
movsbl  -8(%rbp), %eax
movl    %eax, %esi
leaq    .LC0(%rip), %rax

movq    %rax, %rdi
movl    $0, %eax
call    printf@PLT

```

ARM

Аргументи передаються у r0, r1, r2, r3 (EABI).

Адреса .LC0 завантажується через ldr + add, бо ARM не може напряду працювати з далекими адресами

```

.LC0:
ldrb    r3, [fp, #-9] @ zero_extendqisi2
ldrb    r2, [fp, #-10] @ zero_extendqisi2
mov     r1, r3
ldr     r3, .L5

add     r3, pc, r3
mov     r0, r3
bl     printf@PLT

```

ARM64

Аргументи передаються у x0, x1, x2, x3.

Адреса .LC0 рахується через adrp + add - механізм із поділом адреси на старшу й молодшу частину

```

ldrb    w0, [sp, 27]
ldrb    w1, [sp, 26]
mov     w2, w1
mov     w1, w0
adrp    x0, .LC0
add     x0, x0, :lo12:.LC0
bl      printf
}
else {
    hanoi(n-1, from, aux, to);
    printf("Move disk %d from %c to %c\n", n, from, to);
    hanoi(n-1, aux, to, from);
}

```

Різниця з віINDOWS

```

1161: 48 8d 05 9c 0e 00 00    lea     rax,[rip+0xe9c]      # 2004 <_IO_stdin_used+0x4>
1168: 48 89 c7               mov     rdi,rax
116b: b8 00 00 00 00         mov     eax,0x0
1170: e8 bb fe ff ff         call    1030 <printf@plt>
else {

```

У Linux x86-64 виклик printf здійснюється через call printf@plt.

Перед цим перший аргумент передається через регістр RDI (System V ABI), а не через стек, як у Windows x86

```

1157: 0f be 55 f4            movsx   edx,BYTE PTR [rbp-0xc]
115b: 0f be 45 f8            movsx   eax,BYTE PTR [rbp-0x8]
115f: 89 c6                  mov     esi,eax

```

Для завантаження символів у Linux x86-64 використовується інструкція movsx, яка копіює байт у 32-бітний регістр та розширює його зі знаком.

Це відрізняється від Windows x86, де застосовується movzx

```

00000000000011cd <main>:
int main() {
    11cd: 55                push    rbp
    11ce: 48 89 e5          mov     rbp, rsp
    hanoi(3, 'A', 'C', 'B');
    11d1: b9 42 00 00 00    mov     ecx,0x42
    11d6: ba 43 00 00 00    mov     edx,0x43
    11db: be 41 00 00 00    mov     esi,0x41
    11e0: bf 03 00 00 00    mov     edi,0x3
    11e5: e8 4f ff ff ff    call    1139 <hanoi>
}

```

У Linux x86-64 аргументи для виклику функцій передаються через регістри, а не через стек, як у Windows x86.

Тут значення n, from, to, aux завантажуються у EDI, ESI, EDX, ECX перед викликом Hanoi

Linux (LLVM Clang)

```
(kali㉿kali)-[~/lab1/hanoi]
$ clang -O0 -g -o hanoi.clang.amd64 hanoi.c
```

```
(kali㉿kali)-[~/lab1/hanoi]
$ ls
a.out          hanoi.amd64.lst      hanoi.arm          hanoi.arm64.lst    hanoi.arm64.objdump.lst  hanoi.arm.o          hanoi.c
hanoi.amd64    hanoi.amd64.objdump.lst  hanoi.arm64        hanoi.arm64.o      hanoi.arm.lst          hanoi.arm.objdump.lst  hanoi.clang.amd64
```

```
(kali㉿kali)-[~/lab1/hanoi]
$ clang -O0 -g -S -fverbose-asm -o hanoi.clang.amd64.s hanoi.c
```

```
(kali㉿kali)-[~/lab1/hanoi]
$ cat hanoi.clang.amd64.s
.text
.file "hanoi.c"
.file 0 "/home/kali/lab1/hanoi" "hanoi.c" md5 0x02c401d12293a97848f5b69abb35c3bb
.globl hanoi
.p2align 4, 0x90
.type hanoi,@function
hanoi:
.Lfunc_begin0:
.loc 0 3 0
.cfi_startproc
# %bb.0:
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset %rbp, -16
movq %rsp, %rbp
.cfi_def_cfa_register %rbp
subq $16, %rsp
movb %cl, %al
```

функція hanoi приймає аргументи через регістри (%edi для n, %esi/%edx/%ecx для char), зберігає їх на стеку (%rbp, %rsp), перевіряє $n == 1$ через `cmpl` і `jne`, друкує рядки через `printf@PLT` з `leaq` для адрес, і робить рекурсивні виклики через `callq hanoi`. `main` ініціалізує виклик з $n=3$, 'A', 'C', 'B'. `movq/movb` для завантаження, `addq/subq` для стеку, `xorl` для `return 0`, `retq` для повернення. Відмінності від GCC мінімальні, але Clang додає `.cfi` для дебагу. Це типовий x86-64 код із стек-управлінням і динамічними викликами.

```
# %bb.0:
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset %rbp, -16
movq %rsp, %rbp
.cfi_def_cfa_register %rbp
subq $16, %rsp
movb %cl, %al
movb %dl, %cl
movb %sil, %dl
movl %edi, -4(%rbp)
movb %dl, -5(%rbp)
movb %cl, -6(%rbp)
movb %al, -7(%rbp)
```

```

.LBB0_2:
.Ltmp3:
    .loc    0 7 15 is_stmt 1
    movl    -4(%rbp), %edi
    .loc    0 7 16 is_stmt 0
    subl    $1, %edi
    .loc    0 7 20
    movb    -5(%rbp), %cl
    .loc    0 7 26
    movb    -7(%rbp), %al
    .loc    0 7 9
    movsbl  %cl, %esi
    movsbl  %al, %edx
    movsbl  -6(%rbp), %ecx
    callq   hanoi
    .loc    0 8 48 is_stmt 1
    movl    -4(%rbp), %esi
    .loc    0 8 51 is_stmt 0
    movsbl  -5(%rbp), %edx
    .loc    0 8 57
    movsbl  -6(%rbp), %ecx
    .loc    0 8 9
    leaq    .L.str.1(%rip), %rdi
    movb    $0, %al
    callq   printf@PLT

```

2. Реалізувати мовою C/C++, проаналізувати результати компіляції (за варіантом), для платформ i686, amd64, arm, aarch64:

2.1 Комбінаторні алгоритми, будь-який на Ваш вибір з вказаного класу:

Варіант 11

11. на графах– паросполучення (graph matching)

Я обрала Норсфот-Карп, бо він ефективний для максимального паросполучення в біпартиційних (двочасткових) графах. Він включає BFS для шарів і DFS для шляхів, що дозволяє аналізувати цикли, черги та рекурсію в машинному коді.

Реалізація matching.cpp


```

GNU nano 8.4 matching.cpp *
#include <bits/stdc++.h>
using namespace std;

struct HopcroftKarp {
    int nL, nR;
    vector<vector<int>> adj;
    vector<int> dist, pairU, pairV;

    HopcroftKarp(int nL_, int nR_) : nL(nL_), nR(nR_), adj(nL_), pairU(nL_, -1), pairV(nR_, -1), dist(nL_) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v);
    }

    bool bfs() {
        queue<int> q;
        for (int u = 0; u < nL; ++u) {
            if (pairU[u] == -1) {
                dist[u] = 0;
                q.push(u);
            } else {
                dist[u] = INT_MAX;
            }
        }
        bool reachableFree = false;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                int pu = pairV[v];
                if (pu != -1 && dist[pu] == INT_MAX) {
                    dist[pu] = dist[u] + 1;
                    q.push(pu);
                } else if (pu == -1) {
                    reachableFree = true;
                }
            }
        }
        return reachableFree;
    }

    bool dfs(int u) {
        for (int v : adj[u]) {
            int pu = pairV[v];
            if (pu == -1 || (dist[pu] == dist[u] + 1 && dfs(pu))) {

```

File Actions Edit View Help

```

GNU nano 8.4 matching.cpp *
        bool dfs(int u) {
            for (int v : adj[u]) {
                int pu = pairV[v];
                if (pu == -1 || (dist[pu] == dist[u] + 1 && dfs(pu))) {
                    pairU[u] = v;
                    pairV[v] = u;
                    return true;
                }
            }
            dist[u] = INT_MAX;
            return false;
        }

        int maxMatching() {
            int result = 0;
            while (bfs()) {
                for (int u = 0; u < nL; ++u) {
                    if (pairU[u] == -1 && dfs(u)) result++;
                }
            }
            return result;
        }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int nL, nR, m;
    if (!(cin >> nL >> nR >> m)) return 0;
    HopcroftKarp hk(nL, nR);
    for (int i = 0; i < m; ++i) {
        int u, v; cin >> u >> v;
        if (u >= 0 && u < nL && v >= 0 && v < nR) hk.addEdge(u, v);
    }
    int ans = hk.maxMatching();
    cout << ans << "\n";
    for (int u = 0; u < nL; ++u) {
        if (hk.pairU[u] != -1) {
            cout << u << " - " << hk.pairU[u] << "\n";
        }
    }
    return 0;
}

```

Структура зберігає розміри множин (nL , nR), список ребер (adj), відстані ($dist$) і пари ($pairU$, $pairV$). Конструктор ініціалізує вектори. `addEdge` додає ребра. `bfs` шукає шари через чергу (`queue`), позначаючи відстані й перевіряючи вільні вершини. `dfs` рекурсивно

знаходить доповнюючі шляхи, оновлюючи пари. `maxMatching` циклично викликає `bfs` і `dfs`, рахуючи паросполучення. `main` зчитує вхід (`nL`, `nR`, `m`, ребра), створює об'єкт і виводить результат із парами

Компілюю код під `amd64` та запускаю тест, який вводить дані: 3 вершини ліворуч, 3 праворуч, 4 ребра

В виводі бачу спочатку число максимального паросполучення і потім пари

```
(kali㉿kali)-[~/lab1/matching]
$ g++ -O2 -g matching.cpp -o matching.amd64

(kali㉿kali)-[~/lab1/matching]
$ ls -l matching.amd64
-rwxrwxr-x 1 kali kali 178336 Sep  5 16:30 matching.amd64

(kali㉿kali)-[~/lab1/matching]
$ printf "3 3 4\n0 0\n1 1 2\n2 1\n" | ./matching.amd64
3
0 - 0
1 - 2
2 - 1
```

Amd64

```
(kali㉿kali)-[~/lab1/matching]
$ g++ -Wa,-adhln -g matching.cpp > matching.amd64.lst

(kali㉿kali)-[~/lab1/matching]
$ nano matching.amd64.lst

(kali㉿kali)-[~/lab1/matching]
$ objdump -drwC -Intel -S matching.amd64 > matching.amd64.objdump.lst
```

```
528             .section      .text, __ZN12HopcroftKarp3bfsEv, "axG", @progbits, __ZN12HopcroftKarp3bfsEv, comdat
529             .align 2
530             .weak __ZN12HopcroftKarp3bfsEv
532             __ZN12HopcroftKarp3bfsEv:
533             .LFB9844:
14:matching.cpp ****
15:matching.cpp ****      bool bfs() {
534                             .loc 4 15 10
535 0140 24000000             .cfi_startproc
535             8C000000             matching.amd64: matching.amd64: matching.amd64:
535             00000000             00000000
535             A9020000             00000000
535             04000000             00000000
536             .cfi_personality 0x9b,DW.ref.__gxx_personality_v0
537             .cfi_lsda 0x1b,.LLSDA9844
538 0000 55                 pushq   %rbp
539 0155 410E10             .cfi_def_cfa_offset 16
540 0158 8602             .cfi_offset 6, -16
541 0001 4889E5             movq    %rsp, %rbp
542 015a 430D06             .cfi_def_cfa_register 6
543 0004 53                 pushq   %rbx
544 0005 4881ECA8             subq    $168, %rsp
544             00000000
545 015d 488303             .cfi_offset 3, -24
546 000c 4889BD58             movq    %rdi, -168(%rbp)
546             FFFFFFFF
```

pushq %rbp / movq %rsp, %rbp пролог функції

movq %rdi, -168(%rbp) клас HopcroftKarp збережено на стеку

далі буде цикл роботи з чергою , умови, виклики інших функцій STL

I686

```
(kali㉿kali)-[~/lab1/matching]
$ i686-linux-gnu-g++ -O2 -g matching.cpp -o matching.i686

(kali㉿kali)-[~/lab1/matching]
$ i686-linux-gnu-objdump -drwC -Mintel -S matching.i686 > matching.i686.objdump.lst
```

```
00002540 <HopcroftKarp::bfs(>:
    bool bfs() {
2540:      55                push    ebp
2541:      89 e5             mov     ebp,esp
2543:      57                push    edi
2544:      56                push    esi
2545:      53                push    ebx
2546:      e8 05 f2 ff ff    call    1750 <__x86.get_pc_thunk.bx>
254b:      81 c3 a9 2a 00 00    add     ebx,0x2aa9
2551:      81 ec 88 00 00 00    sub     esp,0x88
    : _M_cur(), _M_first(), _M_last(), _M_node() { }
2557:      c7 45 dc 00 00 00 00 mov     DWORD PTR [ebp-0x24],0x0
255e:      8b 75 08             mov     esi,DWORD PTR [ebp+0x8]
2561:      89 5d 94             mov     DWORD PTR [ebp-0x6c],ebx
    return static_cast<_Tp*>(_GLIBCXX_OPERATOR_NEW(__n * sizeof(_Tp)));
2564:      6a 20                push    0x20
2566:      e8 c5 ea ff ff    call    1030 <operator new(unsigned int)@plt>
256b:      89 45 8c             mov     DWORD PTR [ebp-0x74],eax
256e:      89 c7                mov     edi,eax
    _Map_pointer __nstart = (this->_M_impl._M_map
2570:      8d 40 0c             lea     eax,[eax+0xc]
2573:      89 45 98             mov     DWORD PTR [ebp-0x68],eax
2576:      c7 04 24 00 02 00 00 mov     DWORD PTR [esp],0x200
257d:      e8 ae ea ff ff    call    1030 <operator new(unsigned int)@plt>
2582:      83 c4 10             add     esp,0x10
2585:      89 45 b0             mov     DWORD PTR [ebp-0x50],eax
    *__cur = this->_M_allocate_node();
```

push / mov ebp,esp створення стекового фрейму (пролог)

sub esp,0x88 виділення локальних змінних

mov ініціалізація (dist[u] = 0)

далі йдуть виклики STL-функцій (operator new, робота з чергою

Arm, aarch64

```
(kali@kali)-[~/lab1/matching]
$ arm-linux-gnueabi-objdump -drwC -S matching.arm > matching.arm.objdump.lst

(kali@kali)-[~/lab1/matching]
$ aarch64-linux-gnu-objdump -drwC -S matching.aarch64 > matching.aarch64.objdump.lst
```

Arm

```
00001020 <HopcroftKarp::dfs(int)>:
    return *(this->_M_impl._M_start + __n);
1020:     e5902008      ldr     r2, [r0, #8]
bool dfs(int u) {
1024:     e92d4ff0      push   {r4, r5, r6, r7, r8, r9, sl, fp, lr}
1028:     e0813081      add    r3, r1, r1, lsl #1
102c:     e24dd0ac      sub    sp, sp, #172 @ 0xac
1030:     e1a07001      mov    r7, r1
1034:     e58d2008      str    r2, [sp, #8]
1038:     e1a01002      mov    r1, r2
103c:     e0822103      add    r2, r2, r3, lsl #2
1040:     e5922004      ldr    r2, [r2, #4]
1044:     e7913103      ldr    r3, [r1, r3, lsl #2]
1048:     e1a08107      lsl    r8, r7, #2
    for (int v : adj[u]) {
104c:     e1530002      cmp    r3, r2
bool dfs(int u) {
1050:     e58d000c      str    r0, [sp, #12]
1054:     e58d2050      str    r2, [sp, #80] @ 0x50
    for (int v : adj[u]) {
1058:     0a0001de      beq    17d8 <HopcroftKarp::dfs(int)+0x7b8>
105c:     e590e02c      ldr    lr, [r0, #44] @ 0x2c
1060:     e1a09003      mov    r9, r3
    dist[u] = INT_MAX;
1064:     e1a0b00e      mov    fp, lr
1068:     e58d7030      str    r7, [sp, #48] @ 0x30
106c:     ea000002      b      107c <HopcroftKarp::dfs(int)+0x5c>
    for (int v : adj[u]) {
1070:     e59d3050      ldr    r3, [sp, #80] @ 0x50
```

aarch64

```

0000000000001480 <HopcroftKarp::dfs(int)>:
    bool dfs(int u) {
1480:      a9a17bfd      stp     x29, x30, [sp, #-496]!
1484:      aa0003e3      mov     x3, x0
1488:      910003fd      mov     x29, sp
148c:      a9025bf5      stp     x21, x22, [sp, #32]
1490:      2a0103f5      mov     w21, w1
        return *(this->_M_impl._M_start + __n);
1494:      937e7c21      sbfiz   x1, x1, #2, #32
1498:      f9400412      ldr     x18, [x0, #8]
149c:      f90033e0      str     x0, [sp, #96]
14a0:      52800300      mov     w0, #0x18
14a4:      9b207ea0      smull   x0, w21, w0
14a8:      8b000242      add     x2, x18, x0
14ac:      f8606a40      ldr     x0, [x18, x0]
14b0:      f9400442      ldr     x2, [x2, #8]
14b4:      f9006fe2      str     x2, [sp, #216]
        for (int v : adj[u]) {

```

Криптографічні алгоритми, алгоритми кодування та контролю цілісності [26, 27]

11. ChaCha20;

```

GNU nano 8.4                                chacha20.c *
#include <stdint.h>
#include <string.h>
#include <stdio.h>
#include <inttypes.h>

static inline uint32_t ROTL(uint32_t x, int n) {
    return (x << n) | (x >> (32 - n));
}

static inline void quarter_round(uint32_t *a, uint32_t *b, uint32_t *c, uint32_t *d) {
    *a += *b; *d ^= *a; *d = ROTL(*d, 16);
    *c += *d; *b ^= *c; *b = ROTL(*b, 12);
    *a += *b; *d ^= *a; *d = ROTL(*d, 8);
    *c += *d; *b ^= *c; *b = ROTL(*b, 7);
}

void chacha20_block(const uint8_t key[32], const uint8_t nonce[12], uint32_t counter, uint8_t out[64]) {
    const uint32_t constants[4] = { 0x61707865, 0x3320646e, 0x79622d32, 0x6b206574 };
    uint32_t state[16], working[16];
    state[0] = constants[0]; state[1] = constants[1]; state[2] = constants[2]; state[3] = constants[3];
    for (int i = 0; i < 8; ++i) {
        state[4 + i] = ((uint32_t)key[4*i]) | ((uint32_t)key[4*i+1] << 8) | ((uint32_t)key[4*i+2] << 16) | ((uint32_t)key[4*i+3] << 24);
    }
    state[12] = counter;
    state[13] = ((uint32_t)nonce[0]) | ((uint32_t)nonce[1] << 8) | ((uint32_t)nonce[2] << 16) | ((uint32_t)nonce[3] << 24);
    state[14] = ((uint32_t)nonce[4]) | ((uint32_t)nonce[5] << 8) | ((uint32_t)nonce[6] << 16) | ((uint32_t)nonce[7] << 24);
    state[15] = ((uint32_t)nonce[8]) | ((uint32_t)nonce[9] << 8) | ((uint32_t)nonce[10] << 16) | ((uint32_t)nonce[11] << 24);
    memcpy(working, state, sizeof(state));
    for (int i = 0; i < 10; ++i) {
        quarter_round(&working[0], &working[4], &working[8], &working[12]);
        quarter_round(&working[1], &working[5], &working[9], &working[13]);
        quarter_round(&working[2], &working[6], &working[10], &working[14]);
        quarter_round(&working[3], &working[7], &working[11], &working[15]);
        quarter_round(&working[0], &working[5], &working[10], &working[15]);
        quarter_round(&working[1], &working[6], &working[11], &working[12]);
        quarter_round(&working[2], &working[7], &working[8], &working[13]);
        quarter_round(&working[3], &working[4], &working[9], &working[14]);
    }
    for (int i = 0; i < 16; ++i) {
        uint32_t res = working[i] + state[i];
        out[4*i + 0] = (uint8_t)(res & 0xff);
        out[4*i + 1] = (uint8_t)((res >> 8) & 0xff);
        out[4*i + 2] = (uint8_t)((res >> 16) & 0xff);
        out[4*i + 3] = (uint8_t)((res >> 24) & 0xff);
    }
}

```



```

GNU nano 8.4 chacha20.c *
    quarter_round(&working[1], &working[6], &working[11], &working[12]);
    quarter_round(&working[2], &working[7], &working[8], &working[13]);
    quarter_round(&working[3], &working[4], &working[9], &working[14]);
}
for (int i = 0; i < 16; ++i) {
    uint32_t res = working[i] + state[i];
    out[4*i + 0] = (uint8_t)(res & 0xff);
    out[4*i + 1] = (uint8_t)((res >> 8) & 0xff);
    out[4*i + 2] = (uint8_t)((res >> 16) & 0xff);
    out[4*i + 3] = (uint8_t)((res >> 24) & 0xff);
}
}

void chacha20_xor(const uint8_t *key, const uint8_t *nonce, uint32_t counter,
    const uint8_t *in, uint8_t *out, size_t len) {
    size_t i = 0;
    uint8_t keystream[64];
    while (i < len) {
        chacha20_block(key, nonce, counter++, keystream);
        size_t chunk = (len - i) < 64 ? (len - i) : 64;
        for (size_t j = 0; j < chunk; ++j) out[i + j] = in[i + j] ^ keystream[j];
        i += chunk;
    }
}

static void hexdump(const uint8_t *p, size_t n) {
    for (size_t i = 0; i < n; ++i) printf("%02x", p[i]);
    printf("\n");
}

int main(void) {
    uint8_t key[32] = {0};
    uint8_t nonce[12] = {0};
    uint8_t plain[64] = "Hello, ChaCha20 test vector (lab). Use known vectors in report.";
    uint8_t out[128] = {0};
    size_t len = strlen((char*)plain);
    chacha20_xor(key, nonce, 1, plain, out, len);
    printf("ciphertext hex:\n"); hexdump(out, len);
    uint8_t dec[128] = {0};
    chacha20_xor(key, nonce, 1, out, dec, len);
    printf("decrypted: %s\n", dec);
    return 0;
}

```

AMD64

```

(kali㉿kali)-[~/lab1/chacha]
$ nano chacha20.c

(kali㉿kali)-[~/lab1/chacha]
$ gcc -std=c11 -O2 -g chacha20.c -o chacha.amd64

(kali㉿kali)-[~/lab1/chacha]
$ gcc -Wa,-adhln -g chacha20.c > chacha.amd64.lst

```

```

17:chacha20.c      **** void chacha20_block(const uint8_t key[32], const uint8_t nonce[12], uint32_t counter, uint8_t out[6
174                                     .loc 1 17 104
175 0058 1C000000                                     .cfi_startproc
175                                     5C000000
175                                     00000000
175                                     A6040000
175                                     00
176 0133 55                                           pushq   %rbp
177 0069 410E10                                     .cfi_def_cfa_offset 16
178 006c 8602                                     .cfi_offset 6, -16
179 0134 4889E5                                     movq    %rsp, %rbp
180 006e 430D06                                     .cfi_def_cfa_register 6
181 0137 4881ECB0                                     subq    $176, %rsp
181                                     000000
182 013e 4889BD68                                     movq    %rdi, -152(%rbp)
182                                     FFFFFFFF
183 0145 4889B560                                     movq    %rsi, -160(%rbp)
183                                     FFFFFFFF
184 014c 89955CFF                                     movl    %edx, -164(%rbp)
184                                     FFFF
185 0152 4889BD50                                     movq    %rcx, -176(%rbp)
185                                     FFFFFFFF

18:chacha20.c      **** const uint32_t constants[4] = { 0x61707865, 0x3320646e, 0x79622d32, 0x6b206574 };
19:chacha20.c      **** uint32_t state[16], working[16];
20:chacha20.c      **** state[0] = constants[0]; state[1] = constants[1]; state[2] = constants[2]; state[3] = constants

```

I686

```
(kali㉿kali)-[~/lab1/chacha]
$ i686-linux-gnu-gcc -std=c11 -O2 -g chacha20.c -o chacha.i686

(kali㉿kali)-[~/lab1/chacha]
$ i686-linux-gnu-gcc -Wa,-adhln -g chacha20.c > chacha.i686.lst
```

```
16:chacha20.c      ****
17:chacha20.c      **** void chacha20_block(const uint8_t key[32], const uint8_t nonce[12], uint32_t counter, uint8_t out[6]
177                .loc 1 17 104
178 0058 1C000000      .cfi_startproc
178      5C000000
178      07010000
178      0C040000
178      00
179 0107 55          pushl   %ebp
180 0069 410E08      .cfi_def_cfa_offset 8
181 006c 8502      .cfi_offset 5, -8
182 0108 89E5      movl    %esp, %ebp
183 006e 420D05      .cfi_def_cfa_register 5
184 010a 81EC9000      subl    $144, %esp
184      0000
185 0110 E8FCFFFF      call    __x86.get_pc_thunk.ax
185      FF
186 0115 05010000      addl    $_GLOBAL_OFFSET_TABLE_, %eax
186      00
18:chacha20.c      ****      const uint32_t constants[4] = { 0x61707865, 0x3320646e, 0x79622d32, 0x6b206574 };
19:chacha20.c      ****      uint32_t state[16], working[16];
20:chacha20.c      ****      state[0] = constants[0]; state[1] = constants[1]; state[2] = constants[2]; state[3] = constants
187                .loc 1 20 25
```

```
(kali㉿kali)-[~/lab1/chacha]
$ arm-linux-gnueabi-gcc -std=c11 -O2 -g chacha20.c -o chacha.arm

(kali㉿kali)-[~/lab1/chacha]
$ arm-linux-gnueabi-gcc -Wa,-adhln -g chacha20.c > chacha.arm.lst
```

```
16:chacha20.c      ****
17:chacha20.c      **** void chacha20_block(const uint8_t key[32], const uint8_t nonce[12], uint32_t counter, uint8_t out[6]
210                .loc 1 17 104
211                .cfi_startproc
212                @ args = 0, pretend = 0, frame = 160
213                @ frame_needed = 1, uses_anonymous_args = 0
214 01bc 00482DE9      push    {fp, lr}
215                .cfi_def_cfa_offset 8
216                .cfi_offset 11, -8
217                .cfi_offset 14, -4
218 01c0 04B08DE2      add     fp, sp, #4
219                .cfi_def_cfa 11, 4
220 01c4 A0D04DE2      sub     sp, sp, #160
221 01c8 98000BE5      str     r0, [fp, #-152]
222 01cc 9C100BE5      str     r1, [fp, #-156]
223 01d0 A0200BE5      str     r2, [fp, #-160]
224 01d4 A4300BE5      str     r3, [fp, #-164]
18:chacha20.c      ****      const uint32_t constants[4] = { 0x61707865, 0x3320646e, 0x79622d32, 0x6b206574 };
19:chacha20.c      ****      uint32_t state[16], working[16];
20:chacha20.c      ****      state[0] = constants[0]; state[1] = constants[1]; state[2] = constants[2]; state[3] = constants
205                .loc 1 20 25
```

Aarch64

```
(kali㉿kali)-[~/lab1/chacha]
$ aarch64-linux-gnu-gcc -std=c11 -O2 -g chacha20.c -o chacha.aarch64

(kali㉿kali)-[~/lab1/chacha]
$ aarch64-linux-gnu-gcc -Wa,-adhln -g chacha20.c > chacha.aarch64.lst
```



```

10:chacha20.c      ****
17:chacha20.c      **** void chacha20_block(const uint8_t key[32], const uint8_t nonce[12], uint32_t counter, uint8_t out[6
210                .loc 1 17 104
211                .cfi_startproc
212                @ args = 0, pretend = 0, frame = 160
213                @ frame_needed = 1, uses_anonymous_args = 0
214 01bc 00482DE9      push    {fp, lr}
215                .cfi_def_cfa_offset 8
216                .cfi_offset 11, -8
217                .cfi_offset 14, -4
218 01c0 04B08DE2      add     fp, sp, #4
219                .cfi_def_cfa 11, 4
220 01c4 A0D04DE2      sub     sp, sp, #160
221 01c8 98000BE5      str     r0, [fp, #-152]
222 01cc 9C100BE5      str     r1, [fp, #-156]
223 01d0 A0200BE5      str     r2, [fp, #-160]
224 01d4 A4300BE5      str     r3, [fp, #-164]
18:chacha20.c      ****      const uint32_t constants[4] = { 0x61707865, 0x3320646e, 0x79622d32, 0x6b206574 };
19:chacha20.c      ****      uint32_t state[16], working[16];
20:chacha20.c      ****      state[0] = constants[0]; state[1] = constants[1]; state[2] = constants[2]; state[3] = constants
225                .loc 1 20 25

```

Реалізація функцій стандартної бібліотеки C.

11 4. Newlib [31];

Встановлюю крос-компілятор і тулчейн для роботи з Newlib:

```

(kali@kali)-[~/lab1/newlib]
$ sudo apt update
sudo apt install gcc-arm-none-eabi gdb-multiarch build-essential

[sudo] password for kali:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
968 packages can be upgraded. Run 'apt list --upgradable' to see them.
gcc-arm-none-eabi is already the newest version (15:14.2.rel1-1).
build-essential is already the newest version (12.12).
The following packages were automatically installed and are no longer required:

```

Завантажую Newlib

```

(kali@kali)-[~/lab1/newlib]
$ wget ftp://sourceware.org/pub/newlib/newlib-4.3.0.20230120.tar.gz
tar -xvzf newlib-4.3.0.20230120.tar.gz
--2025-09-20 13:16:21-- ftp://sourceware.org/pub/newlib/newlib-4.3.0.20230120.tar.gz
      => 'newlib-4.3.0.20230120.tar.gz'
Resolving sourceware.org (sourceware.org)... 8.43.85.97, 2620:52:3:1:0:246e:9693:128c
Connecting to sourceware.org (sourceware.org)|8.43.85.97|:21 ... connected.
Logging in as anonymous ... Logged in!
=> SYST ... done.      => PWD ... done.
=> TYPE I ... done.    => CWD (1) /pub/newlib ... done.
=> SIZE newlib-4.3.0.20230120.tar.gz ... 8832922
=> PASV ... done.      => RETR newlib-4.3.0.20230120.tar.gz ... done.
Length: 8832922 (8.4M) (unauthoritative)

newlib-4.3.0.20230120.tar.gz      100%[====>] 8.42M 2.81MB/s in 3.0s
2025-09-20 13:16:26 (2.81 MB/s) - 'newlib-4.3.0.20230120.tar.gz' saved [8832922]

```

Створюю скрипт, який використовує потрібні функції стандартного ввід-виводу printf, puts

```
GNU nano 8.4
#include <stdio.h>

int main(void) {
    printf("printf: %d %s\n", 42, "hello");
    puts("puts: hello world");
    return 0;
}
```

роботи з файлами fopen, fread, fwrite, feof, fclose

```
#include <stdio.h>
#include <string.h>

int main(void) {
    const char *fn = "test.bin";
    FILE *f = fopen(fn, "wb");
    const char *s = "ABCDEFGH";
    fwrite(s, 1, strlen(s), f);
    fclose(f);

    f = fopen(fn, "rb");
    char buf[32];
    size_t n = fread(buf, 1, sizeof(buf), f);
    buf[n] = 0;
    if (!feof(f)) {
        puts("not EOF yet");
    }
    fclose(f);
    puts(buf);
    return 0;
}
```

виконання команд операційної системи system.

```
GNU nano 8.4
#include <stdlib.h>
#include <stdio.h>

int main(void) {
    int r = system("echo hello_from_system");
    printf("system() returned %d\n", r);
    return 0;
}
```


Syscalls.c для мінімальної реалізації цих функцій

```
GNU nano 8.4
#include <sys/stat.h>
#include <unistd.h>
#include <errno.h>
#include <stddef.h>

int _write(int file, const char *ptr, int len) {
    return len;
}

int _read(int file, char *ptr, int len) {
    return 0;
}

int _close(int file) {
    return -1;
}

int _lseek(int file, int ptr, int dir) {
    return 0;
}

int _fstat(int file, struct stat *st) {
    st->st_mode = S_IFCHR;
    return 0;
}

int _isatty(int file) {
    return 1;
}

void *_sbrk(ptrdiff_t incr) {
    extern char _end;
    static char *heap_end;
    char *prev_heap_end;

    if (heap_end == 0) {
        heap_end = &_end;
    }
    prev_heap_end = heap_end;
    heap_end += incr;
    return (void *) prev_heap_end;
}

int _getpid(void) {
```

ARM

Компіляція та лістинги(arm-none-eabi-gcc автоматично лінкується Newlib)

```
(kali㉿kali)-[~/lab1/newlib]
$ arm-none-eabi-gcc -O0 -g -o stdio_arm.elf stdio.c syscalls.c
arm-none-eabi-gcc -O0 -g -o files_arm.elf files.c syscalls.c
arm-none-eabi-gcc -O0 -g -o system_arm.elf system.c syscalls.c
```

```
(kali㉿kali)-[~/lab1/newlib]
$ arm-none-eabi-objdump -d stdio_arm.elf > stdio_arm.lst
arm-none-eabi-objdump -d files_arm.elf > files_arm.lst
arm-none-eabi-objdump -d system_arm.elf > system_arm.lst
```

Для stdio:

```
0000824c <main>:
824c: e92d4800 push {fp, lr}
8250: e28db004 add fp, sp, #4
8254: e59f2024 ldr r2, [pc, #36] @ 8280 <main+0x34>
8258: e3a0102a mov r1, #42 @ 0x2a
825c: e59f0020 ldr r0, [pc, #32] @ 8284 <main+0x38>
8260: eb00022e bl 8b20 <printf>
8264: e59f001c ldr r0, [pc, #28] @ 8288 <main+0x3c>
8268: eb0000f6 bl 8648 <puts>
826c: e3a03000 mov r3, #0
8270: e1a00003 mov r0, r3
8274: e24bd004 sub sp, fp, #4
8278: e8bd4800 pop {fp, lr}
827c: e12fff1e bx lr
8280: 000123d4 .word 0x000123d4
8284: 000123dc .word 0x000123dc
8288: 000123ec .word 0x000123ec
```

У ARM виклики функцій передають аргументи через регістри r0–r3. У mov r1, #42 число 42 поміщається в r1. ldr r0, [...] завантажує адресу рядка у r0. Інструкція bl printf викликає функцію printf. Аналогічно потім викликається puts із аргументом у r0

```
00008b20 <printf>:
8b20: e92d000f push {r0, r1, r2, r3}
8b24: e52de004 push {lr} @ (str lr, [sp, #-4]!)
)
8b28: e59f2028 ldr r2, [pc, #40] @ 8b58 <printf+0x38>
8b2c: e5920000 ldr r0, [r2]
8b30: e24dd00c sub sp, sp, #12
8b34: e28d3014 add r3, sp, #20
8b38: e59d2010 ldr r2, [sp, #16]
8b3c: e5901008 ldr r1, [r0, #8]
8b40: e58d3004 str r3, [sp, #4]
8b44: eb00058b bl a178 <_vfprintf_r>
8b48: e28dd00c add sp, sp, #12
8b4c: e49de004 pop {lr} @ (ldr lr, [sp], #4)
8b50: e28dd010 add sp, sp, #16
8b54: e12fff1e bx lr
8b58: 00022bd0 .word 0x00022bd0
```

Видно, що printf не реалізує форматування напряму, а викликає внутрішню функцію _vfprintf_r

```

0000861c <_printf_r>:
 861c: e92d000e      push    {r1, r2, r3}
 8620: e52de004      push    {lr}           @ (str lr, [sp, #-4]!)
 8624: e24dd008      sub     sp, sp, #8
 8628: e28d3010      add     r3, sp, #16
 862c: e59d200c      ldr     r2, [sp, #12]
 8630: e5901008      ldr     r1, [r0, #8]
 8634: e58d3004      str     r3, [sp, #4]
 8638: eb00004e      bl      8778 <_vfprintf_r>
 863c: e28dd008      add     sp, sp, #8
 8640: e49de004      pop     {lr}           @ (ldr lr, [sp], #4)
 8644: e28dd00c      add     sp, sp, #12
 8648: e12fff1e      bx      lr

```

```

00008588 <_puts_r>:
 8588: e92d4030      push    {r4, r5, lr}
 858c: e1a04000      mov     r4, r0
 8590: e24dd02c      sub     sp, sp, #44     @ 0x2c
 8594: e1a00001      mov     r0, r1
 8598: e1a05001      mov     r5, r1
 859c: eb0006dd      bl      a118 <strlen>
 85a0: e3a02001      mov     r2, #1
 85a4: e3a03002      mov     r3, #2
 85a8: e59f1094      ldr     r1, [pc, #148]   @ 8644 <_puts_r+0xbc>
 85ac: e58d1020      str     r1, [sp, #32]
 85b0: e5941034      ldr     r1, [r4, #52]    @ 0x34
 85b4: e58d2024      str     r2, [sp, #36]    @ 0x24
 85b8: e58d3010      str     r3, [sp, #16]
 85bc: e28d2018      add     r2, sp, #24
 85c0: e3510000      cmp     r1, #0
 85c4: e2803001      add     r3, r0, #1
 85c8: e58d5018      str     r5, [sp, #24]
 85cc: e58d200c      str     r2, [sp, #12]
 85d0: e5941008      ldr     r1, [r4, #8]
 85d4: e58d001c      str     r0, [sp, #28]
 85d8: e58d3014      str     r3, [sp, #20]
 85dc: 0a000011      beq     8628 <_puts_r+0xa0>
 85e0: e1d120fc      ldrsh   r2, [r1, #12]
 85e4: e5913064      ldr     r3, [r1, #100]   @ 0x64
 85e8: e3120a02      tst     r2, #8192        @ 0x2000
 85ec: 03822a02      orreq   r2, r2, #8192    @ 0x2000

```

У `_printf_r` видно, що функція лише готує параметри (збереження регістрів, підготовка вказівників) і викликає `_vfprintf_r`, де виконується основне форматування та друк. Після повернення очищається стек і здійснюється вихід — тобто це тонка обгортка навколо справжнього механізму форматованого виводу. У `_puts_r` спочатку викликається `strlen` для визначення довжини рядка, далі налаштовуються структури потоку і прапорці, за потреби виконується ініціалізація потоків через `__sinit`. Основний запис виконується викликом `__sfvwrite_r`, після чого додається символ нового рядка. У разі помилки повертається -1. Таким чином, `_puts_r` забезпечує підготовку даних, запис рядка через внутрішню функцію і автоматичне додавання `\n` наприкінці.

Для files:

```

00009d30 <_fopen_r>:
9d30: e1a0c002    mov     ip, r2
9d34: e92d40f0    push   {r4, r5, r6, r7, lr}
9d38: e24d000c    sub     sp, sp, #12
9d3c: e1a07001    mov     r7, r1
9d40: e28d2004    add     r2, sp, #4
9d44: e1a0100c    mov     r1, ip
9d48: e1a06000    mov     r6, r0
9d4c: eb0003d9    bl      acb8 <__sflags>
9d50: e2505000    subs    r5, r0, #0
9d54: 0a00002a    beq     9e04 <_fopen_r+0xd4>
9d58: e1a00006    mov     r0, r6
9d5c: ebfffbf9    bl      8d4c <__sfp>
9d60: e2504000    subs    r4, r0, #0
9d64: 0a000026    beq     9e04 <_fopen_r+0xd4>
9d68: e1a01007    mov     r1, r7
9d6c: e1a00006    mov     r0, r6
9d70: e59f3094    ldr     r3, [pc, #148] @ 9e0c <_fopen_r+0xdc>
9d74: e59d2004    ldr     r2, [sp, #4]
9d78: eb00049f    bl      affc <_open_r>
9d7c: e3500000    cmp     r0, #0
9d80: ba00001b    blt     9df4 <_fopen_r+0xc4>
9d84: e1a03805    lsl     r3, r5, #16
9d88: e1a03843    asr     r3, r3, #16
9d8c: e1c430bc    strh    r3, [r4, #12]
9d90: e3130c01    tst     r3, #256 @ 0x100
9d94: e59f3074    ldr     r3, [pc, #116] @ 9e10 <_fopen_r+0xe0>
9d98: e59f1074    ldr     r1, [pc, #116] @ 9e14 <_fopen_r+0xe4>
9d9c: e59f2074    ldr     r2, [pc, #116] @ 9e18 <_fopen_r+0xe8>
9da0: e5843024    str     r3, [r4, #36] @ 0x24
9da4: e59f3070    ldr     r3, [pc, #112] @ 9e1c <_fopen_r+0xec>
9da8: e1c400be    strh    r0, [r4, #14]
9dac: e584401c    str     r4, [r4, #28]
9db0: e5841020    str     r1, [r4, #32]
9db4: e5842028    str     r2, [r4, #40] @ 0x28
9db8: e584302c    str     r3, [r4, #44] @ 0x2c
9dbc: 1a000003    bne     9dd0 <_fopen_r+0xa0>
9dc0: e1a00004    mov     r0, r4
9dc4: e28dd00c    add     sp, sp, #12
9dc8: e8bd40f0    pop     {r4, r5, r6, r7, lr}
9dcc: e12ffff1e    bx      lr
9dd0: e3a03002    mov     r3, #2
9dd4: e3a02000    mov     r2, #0
9dd8: e1a01004    mov     r1, r4
9ddc: e1a00006    mov     r0, r6

```

Аргументи зберігаються в регістри (r0, r1, r2 → далі працюють із ними). Викликається допоміжна функція `__sflags` для розбору режиму відкриття файлу ("r", "w", "wb+" тощо). Викликається `__sfp` — виділити структуру FILE у newlib. виклик `_open_r` (рядок 9d78: `bl affc <_open_r>`)це і є перехід на низькорівневу функцію `_open`, яку надає юзер у `syscalls.c`

```

00008768 <_fread_r>:
8768: e92d4ff0    push   {r4, r5, r6, r7, r8, r9, sl, fp, lr}
876c: e24d000c    sub     sp, sp, #12
8770: e58d2000    str     r2, [sp]
8774: e0120293    muls    r2, r3, r2
8778: e59d4030    ldr     r4, [sp, #48] @ 0x30
877c: 0a00003b    beq     8870 <_fread_r+0x108>
8780: e3500000    cmp     r0, #0
8784: e1a07003    mov     r7, r3
8788: e1a06000    mov     r6, r0
878c: e1a05001    mov     r5, r1
8790: e1a0a002    mov     sl, r2
8794: 0a000002    beq     87a4 <_fread_r+0x3c>
8798: e5903034    ldr     r3, [r0, #52] @ 0x34
879c: e3530000    cmp     r3, #0
87a0: 0a000036    beq     8880 <_fread_r+0x118>
87a4: e1d420fc    ldrsh   r2, [r4, #12]
87a8: e5943064    ldr     r3, [r4, #100] @ 0x64
87ac: e3120a02    tst     r2, #8192 @ 0x2000
87b0: 03c33a02    biceq   r3, r3, #8192 @ 0x2000
87b4: 05843064    streq   r3, [r4, #100] @ 0x64
87b8: 03822a02    orreq   r2, r2, #8192 @ 0x2000
87bc: e1a036c3    asr     r3, r3, #13
87c0: 01c420bc    strheq  r2, [r4, #12]
87c4: e2133001    ands    r3, r3, #1
87c8: 1a000028    bne     8870 <_fread_r+0x108>
87cc: e5948004    ldr     r8, [r4, #4]
87d0: e1d420bc    ldrh    r2, [r4, #12]
87d4: e3580000    cmp     r8, #0
87d8: a1a03008    movge   r3, r8
87dc: b1a08003    movlt   r8, r3
87e0: b5843004    strlt   r3, [r4, #4]
87e4: e3120002    tst     r2, #2
87e8: 01a0b00a    moveq   fp, sl
87ec: e5941000    ldr     r1, [r4]
87f0: 0a00000f    beq     8834 <_fread_r+0xcc>
87f4: ea000023    b       8888 <_fread_r+0x120>
87f8: e1a02008    mov     r2, r8
87fc: e1a00005    mov     r0, r5
8800: eb000591    bl      9e4c <memcpy>
8804: e5943000    ldr     r3, [r4]
8808: e0833008    add     r3, r3, r8
880c: e1a01004    mov     r1, r4
8810: e1a00006    mov     r0, r6
8814: e5843000    str     r3, [r4]

```



```

000089e8 <_fclose_r>:
89e8: e92d4070    push    {r4, r5, r6, lr}
89ec: e2514000    subs    r4, r1, #0
89f0: 0a000008    beq     8a18 <_fclose_r+0x30>
89f4: e3500000    cmp     r0, #0
89f8: e1a05000    mov     r5, r0
89fc: 0a000002    beq     8a0c <_fclose_r+0x24>
8a00: e5903034    ldr     r3, [r0, #52] @ 0x34
8a04: e3530000    cmp     r3, #0
8a08: 0a00002d    beq     8ac4 <_fclose_r+0xdc>
8a0c: e1d430fc    ldrsh   r3, [r4, #12]
8a10: e3530000    cmp     r3, #0
8a14: 1a000003    bne     8a28 <_fclose_r+0x40>
8a18: e3a06000    mov     r6, #0
8a1c: e1a00006    mov     r0, r6
8a20: e8bd4070    pop     {r4, r5, r6, lr}
8a24: e12fff1e    bx      lr
8a28: e1a01004    mov     r1, r4
8a2c: e1a00005    mov     r0, r5
8a30: eb0004bc    bl      9d28 <__sflush_r>
8a34: e594302c    ldr     r3, [r4, #44] @ 0x2c
8a38: e3530000    cmp     r3, #0
8a3c: e1a06000    mov     r6, r0
8a40: 0a000005    beq     8a5c <_fclose_r+0x74>
8a44: e1a00005    mov     r0, r5
8a48: e594101c    ldr     r1, [r4, #28]
8a4c: e1a0e00f    mov     lr, pc
8a50: e12fff13    bx      r3
8a54: e3500000    cmp     r0, #0
8a58: b3e06000    mvnlt   r6, #0
8a5c: e1d430bc    ldrh    r3, [r4, #12]
8a60: e3130080    tst     r3, #128 @ 0x80
8a64: 1a000018    bne     8acc <_fclose_r+0xe4>
8a68: e5941030    ldr     r1, [r4, #48] @ 0x30
8a6c: e3510000    cmp     r1, #0
8a70: 0a000005    beq     8a8c <_fclose_r+0xa4>
8a74: e2843040    add     r3, r4, #64 @ 0x40
8a78: e1510003    cmp     r1, r3
8a7c: 11a00005    movne   r0, r5
8a80: 1b00025a    blne    93f0 <_free_r>
8a84: e3a03000    mov     r3, #0
8a88: e5843030    str     r3, [r4, #48] @ 0x30
8a8c: e5941044    ldr     r1, [r4, #68] @ 0x44
8a90: e3510000    cmp     r1, #0
8a94: 0a000003    beq     8aa8 <_fclose_r+0xc0>

```

У фрагменті `_fread_r` видно, що бібліотека спочатку намагається задовольнити читання з внутрішнього буфера, використовуючи інструкцію `bl memscr`. Якщо буфер порожній, викликаються нижчі рівні вводу-виводу через `_read`.

У `_fclose_r` перед закриттям файлу відбувається скидання буфера (`__sflush_r`), виклик функції закриття, яка прив'язана до потоку, та звільнення пам'яті.

Для system:

```

00008588 <_system_r>:
8588: e2510000    subs    r0, r1, #0
858c: 012fff1e    bxeq    lr
8590: e92d4010    push    {r4, lr}
8594: eb00003b    bl      8688 <__errno>
8598: e3a02058    mov     r2, #88 @ 0x58
859c: e1a03000    mov     r3, r0
85a0: e8bd4010    pop     {r4, lr}
85a4: e3e00000    mvn     r0, #0
85a8: e5832000    str     r2, [r3]
85ac: e12fff1e    bx      lr

```

У `_system_r` бачимо stub-реалізацію: функція одразу повертає -1, а змінній `errno` присвоюється значення 88 (ENOSYS), що означає «системний виклик не реалізований»

Для i686 та x86_64 завантажую тулчейни, потім розпаковую

```
(kali㉿kali)-[~/toolchains]
$ wget -O i686-elf-tools-linux.zip \
  "https://github.com/lordmilkoi686-elf-tools/releases/download/7.1.0/i686-elf-tools-linux.zip"
--2025-09-21 04:15:00-- https://github.com/lordmilkoi686-elf-tools/releases/download/7.1.0/i686-elf-tools-linux.zip
Resolving github.com (github.com)... 140.82.121.3
Connecting to github.com (github.com)|140.82.121.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release-asset/47872348/b67a3460-8a58-11e7-9826-78978abf8e95?sp=r&sv=2018-11-09&sr=b&spr=https&se=2025-09-21T08%3A53%3A19Z&rsct=attachment%3B+filename%3Di686-elf-tools-linux.zip&rsct=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7a2b&sig=V80Vb8H3WKL0QJDeV75BjWBovN4z1JGYiwenitMWqNA%3D&wt=ev10eXA104JKY101LC1h1
```

```
(kali㉿kali)-[~/toolchains]
$ wget -O x86_64-elf-tools-linux.zip \
  "https://github.com/lordmilkoi686-elf-tools/releases/download/7.1.0/x86_64-elf-tools-linux.zip"
--2025-09-21 04:19:32-- https://github.com/lordmilkoi686-elf-tools/releases/download/7.1.0/x86_64-elf-tools-linux.zip
Resolving github.com (github.com)... 140.82.121.3
Connecting to github.com (github.com)|140.82.121.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release-asset/47872348/ef458880-712d-11ea-8544-046ttachment%3B+filename%3Dx86_64-elf-tools-linux.zip&rsct=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7a2b&sig=V80Vb8H3WKL0QJDeV75BjWBovN4z1JGYiwenitMWqNA%3D&wt=ev10eXA104JKY101LC1h1
```

Бачу що все працює

```
(kali㉿kali)-[~/toolchains]
$ i686-elf-gcc --version
x86_64-elf-gcc --version
i686-elf-objdump --version
x86_64-elf-objdump --version

i686-elf-gcc (GCC) 7.1.0
Copyright (C) 2017 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

x86_64-elf-gcc (GCC) 7.1.0
Copyright (C) 2017 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

GNU objdump (GNU Binutils) 2.28
Copyright (C) 2017 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or (at your option) any later version.
This program has absolutely no warranty.
GNU objdump (GNU Binutils) 2.28
Copyright (C) 2017 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or (at your option) any later version.
This program has absolutely no warranty.
```

I686

Компіляція та лістинг

```

(kali㉿kali)-[~/lab1/newlib]
$ i686-elf-gcc -O0 -g -nostartfiles \
-L/home/kali/lab1/newlib/install-i686/lib \
-I/home/kali/lab1/newlib/install-i686/include \
-o stdio_i686.elf stdio.c syscalls.o -lc -lgcc

i686-elf-gcc -O0 -g -nostartfiles \
-L/home/kali/lab1/newlib/install-i686/lib \
-I/home/kali/lab1/newlib/install-i686/include \
-o files_i686.elf files.c syscalls.o -lc -lgcc

i686-elf-gcc -O0 -g -nostartfiles \
-L/home/kali/lab1/newlib/install-i686/lib \
-I/home/kali/lab1/newlib/install-i686/include \
-o system_i686.elf system.c syscalls.o -lc -lgcc

```

```

(kali㉿kali)-[~/lab1/newlib]
$ i686-elf-objdump -d -M intel stdio_i686.elf > stdio_i686.lst
i686-elf-objdump -d -M intel files_i686.elf > files_i686.lst
i686-elf-objdump -d -M intel system_i686.elf > system_i686.lst

```

Для stdio

```

08048080 <main>:
8048080: 8d 4c 24 04      lea    ecx,[esp+0x4]
8048084: 83 e4 f0 2a 0c   and    esp,0xffffffff
8048087: ff 71 fc        push   DWORD PTR [ecx-0x4]
804808a: 55             push   ebp
804808b: 89 e5 2a 0c     mov    ebp,esp
804808d: 51             push   ecx
804808e: 83 ec 04        sub    esp,0x4
8048091: 83 ec 04        sub    esp,0x4
8048094: 68 80 19 05 08   push   0x8051980
8048099: 6a 2a          push   0x2a
804809b: 68 86 19 05 08   push   0x8051986
80480a0: e8 1b 01 00 00   call   80481c0 <printf>
80480a5: 83 c4 10        add    esp,0x10
80480a8: 83 ec 0c        sub    esp,0xc
80480ab: 68 95 19 05 08   push   0x8051995
80480b0: e8 cb 01 00 00   call   8048280 <puts>
80480b5: 83 c4 10        add    esp,0x10
80480b8: b8 00 00 00 00   mov    eax,0x0
80480bd: 8b 4d fc        mov    ecx,DWORD PTR [ebp-0x4]
80480c0: c9             leave  esp
80480c1: 8d 61 fc        lea    esp,[ecx-0x4]
80480c4: c3             ret

```

```

080481a0 <_printf_r>:
80481a0: 83 ec 0c          sub     esp,0xc
80481a3: 8b 44 24 10       mov     eax,DWORD PTR [esp+0x10]
80481a7: 8d 54 24 18       lea     edx,[esp+0x18]
80481ab: 52              push    edx
80481ac: ff 74 24 18       push   DWORD PTR [esp+0x18]
80481b0: ff 70 08          push   DWORD PTR [eax+0x8]
80481b3: 50              push    eax
80481b4: e8 77 01 00 00    call   8048330 <_vfprintf_r>
80481b9: 83 c4 1c          add     esp,0x1c
80481bc: c3              ret
80481bd: 8d 76 00          lea     esi,[esi+0x0]

080481c0 <printf>:
80481c0: 83 ec 0c          sub     esp,0xc
80481c3: a1 00 57 05 08    mov     eax,ds:0x8055700
80481c8: 8d 54 24 14       lea     edx,[esp+0x14]
80481cc: 52              push    edx
80481cd: ff 74 24 14       push   DWORD PTR [esp+0x14]
80481d1: ff 70 08          push   DWORD PTR [eax+0x8]
80481d4: 50              push    eax
80481d5: e8 56 01 00 00    call   8048330 <_vfprintf_r>
80481da: 83 c4 1c          add     esp,0x1c
80481dd: c3              ret
80481de: 66 90          xchg    ax,ax

080481e0 <_puts_r>:
80481e0: 56              push    esi
80481e1: 53              push    ebx
80481e2: 83 ec 30          sub     esp,0x30
80481e5: 8b 5c 24 40       mov     ebx,DWORD PTR [esp+0x40]
80481e9: 8b 74 24 3c       mov     esi,DWORD PTR [esp+0x3c]
80481ed: 53              push    ebx
80481ee: e8 bd 00 00 00    call   80482b0 <strlen>
80481f3: 89 44 24 24       mov     DWORD PTR [esp+0x24],eax
80481f7: 83 c0 01          add     eax,0x1
80481fa: 89 5c 24 20       mov     DWORD PTR [esp+0x20],ebx
80481fe: 89 44 24 1c       mov     DWORD PTR [esp+0x1c],eax
8048202: 8d 44 24 20       lea     eax,[esp+0x20]
8048206: c7 44 24 28 a7 19 05 mov     DWORD PTR [esp+0x28],0x80519a7
804820d: 08
804820e: c7 44 24 2c 01 00 00 mov     DWORD PTR [esp+0x2c],0x1
8048215: 00
8048216: c7 44 24 18 02 00 00 mov     DWORD PTR [esp+0x18],0x2
804821d: 00

```

У `_printf_r` аргументи кладуться у стек і передаються у внутрішню функцію `_vfprintf_r`, яка робить усе форматування. У `_puts_r` спочатку викликається `strlen`, далі формується структура параметрів і дані виводяться через `__sfvwrite_r`. У `main` видно підготовку аргументів у стек, виклик `printf`, а потім `puts`.

Для files:


```

08048080 <main>:
8048080: 8d 4c 24 04      lea     ecx,[esp+0x4]
8048084: 83 e4 f0        and     esp,0xfffffff0
8048087: ff 71 fc        push   DWORD PTR [ecx-0x4]
804808a: 55             push   ebp
804808b: 89 e5          mov     ebp,esp
804808d: 51             push   ecx
804808e: 83 ec 34        sub     esp,0x34
8048091: c7 45 f4 09 b8 04 08 mov     DWORD PTR [ebp-0xc],0x804b809
8048098: 83 ec 08        sub     esp,0x8
804809b: 68 12 b8 04 08  push   0x804b812
80480a0: ff 75 f4        push   DWORD PTR [ebp-0xc]
80480a3: e8 b8 0a 00 00  call    8048b60 <fopen>
80480a8: 83 c4 10        add     esp,0x10
80480ab: 89 45 f0        mov     DWORD PTR [ebp-0x10],eax
80480ae: c7 45 ec 15 b8 04 08 mov     DWORD PTR [ebp-0x14],0x804b815
80480b5: 83 ec 0c        sub     esp,0xc
80480b8: ff 75 ec        push   DWORD PTR [ebp-0x14]
80480bb: e8 10 1b 00 00  call    8049bd0 <strlen>
80480c0: 83 c4 10        add     esp,0x10
80480c3: ff 75 f0        push   DWORD PTR [ebp-0x10]
80480c6: 50             push   eax
80480c7: 6a 01          push   0x1
80480c9: ff 75 ec        push   DWORD PTR [ebp-0x14]
80480cc: e8 ff 0d 00 00  call    8048ed0 <fwrite>
80480d1: 83 c4 10        add     esp,0x10
80480d4: 83 ec 0c        sub     esp,0xc
80480d7: ff 75 f0        push   DWORD PTR [ebp-0x10]
80480da: e8 61 02 00 00  call    8048340 <fclose>
80480df: 83 c4 10        add     esp,0x10
80480e2: 83 ec 08        sub     esp,0x8
80480e5: 68 1d b8 04 08  push   0x804b81d
80480ea: ff 75 f4        push   DWORD PTR [ebp-0xc]
80480ed: e8 6e 0a 00 00  call    8048b60 <fopen>
80480f2: 83 c4 10        add     esp,0x10
80480f5: 89 45 f0        mov     DWORD PTR [ebp-0x10],eax
80480f8: ff 75 f0        push   DWORD PTR [ebp-0x10]
80480fb: 6a 20          push   0x20
80480fd: 6a 01          push   0x1
80480ff: 8d 45 c8        lea     eax,[ebp-0x38]
8048102: 50             push   eax
8048103: e8 68 0c 00 00  call    8048d70 <fread>
8048108: 83 c4 10        add     esp,0x10
804810b: 89 45 e8        mov     DWORD PTR [ebp-0x18],eax
804810e: 8d 55 c8        lea     edx,[ebp-0x38]

```

```

08048a90 <_fopen_r>:
8048a90: 57             push   edi
8048a91: 56             push   esi
8048a92: 53             push   ebx
8048a93: 83 ec 14        sub     esp,0x14
8048a96: 8b 74 24 24     mov     esi,DWORD PTR [esp+0x24]
8048a9a: 8d 44 24 10     lea     eax,[esp+0x10]
8048a9e: 50             push   eax
8048a9f: ff 74 24 30     push   DWORD PTR [esp+0x30]
8048aa3: 56             push   esi
8048aa4: e8 f7 1d 00 00  call    804a8a0 <__sflags>
8048aa9: 83 c4 10        add     esp,0x10
8048aac: 85 c0          test    eax,eax
8048aae: 0f 84 7c 00 00 00 je      8048b30 <_fopen_r+0xa0>
8048ab4: 83 ec 0c        sub     esp,0xc
8048ab7: 89 c7          mov     edi,eax
8048ab9: 56             push   esi
8048aba: e8 11 fe ff ff  call    80488d0 <__sfp>
8048abf: 83 c4 10        add     esp,0x10
8048ac2: 85 c0          test    eax,eax
8048ac4: 89 c3          mov     ebx,eax

```

У main відкривається файл через fopen, далі виконується fwrite для запису рядка, після чого файл закривається. Потім файл відкривається знову у режимі читання, дані зчитуються у буфер через fread, у кінець додається нуль-байт, виконується перевірка feof і вивід через puts. У середині _fopen_r видно виклики __sflags, __sfp та _open_r, які реалізують системний рівень роботи з файлами.

Для system:

```

08048080 <main>:
8048080: 8d 4c 24 04      lea     ecx,[esp+0x4]
8048084: 83 e4 f0 00 00   and     esp,0xffffffff
8048087: ff 71 fc         push   DWORD PTR [ecx-0x4]
804808a: 55              push   ebp
804808b: 89 e5           mov     ebp,esp
804808d: 51              push   ecx
804808e: 83 ec 14        sub     esp,0x14
8048091: 83 ec 0c        sub     esp,0xc
8048094: 68 20 19 05 08   push   0x8051920
8048099: e8 42 01 00 00   call   80481e0 <system>
804809e: 83 c4 10        add     esp,0x10
80480a1: 89 45 f4        mov     DWORD PTR [ebp-0xc],eax
80480a4: 83 ec 08        sub     esp,0x8
80480a7: ff 75 f4        push   DWORD PTR [ebp-0xc]
80480aa: 68 37 19 05 08   push   0x8051937
80480af: e8 7c 01 00 00   call   8048230 <printf>
80480b4: 83 c4 10        add     esp,0x10
80480b7: b8 00 00 00 00   mov     eax,0x0
80480bc: 8b 4d fc        mov     ecx,DWORD PTR [ebp-0x4]
80480bf: c9              leave   esp
80480c0: 8d 61 fc        lea     esp,[ecx-0x4]
80480c3: c3              ret

```

```

80481a0 <_system_r>:
80481a0: 83 ec 0c        sub     esp,0xc
80481a3: 8b 44 24 14 0c   mov     eax,DWORD PTR [esp+0x14]
80481a7: 85 c0           test    eax,eax
80481a9: 75 15           jne     80481c0 <_system_r+0x20>
80481ab: 31 c0           xor     eax,eax
80481ad: 83 c4 0c        add     esp,0xc
80481b0: c3              ret
80481b1: eb 0d           jmp     80481c0 <_system_r+0x20>
80481b3: 90              nop
80481b4: 90              nop
80481b5: 90              nop
80481b6: 90              nop
80481b7: 90              nop
80481b8: 90              nop
80481b9: 90              nop
80481ba: 90              nop
80481bb: 90              nop
80481bc: 90              nop
80481bd: 90              nop
80481be: 90              nop
80481bf: 90              nop
80481c0: e8 8b 00 00 00   call   8048250 <__errno>
80481c5: c7 00 58 00 00 00 mov     DWORD PTR [eax],0x58
80481cb: b8 ff ff ff ff   mov     eax,0xffffffff
80481d0: eb db           jmp     80481ad <_system_r+0xd>
80481d2: 8d b4 26 00 00 00 lea     esi,[esi+eiz*1+0x0]
80481d9: 8d bc 27 00 00 00 lea     edi,[edi+eiz*1+0x0]

```

У main у стек передається рядок з командою, викликається system, результат записується у змінну і друкується через printf. Усередині _system_r йде перевірка: якщо аргумент NULL, повертається 0; якщо ні – встановлюється помилка ENOSYS і повертається -1

X86_64

```

x86_64-elf-gcc -O0 -g -nostartfiles -L"$LIBDIR" -I"$INC_DIR" \
-o stdio_x86_64.elf stdio.c syscalls_x86_64.o -lc -lgcc

x86_64-elf-gcc -O0 -g -nostartfiles -L"$LIBDIR" -I"$INC_DIR" \
-o files_x86_64.elf files.c syscalls_x86_64.o -lc -lgcc

x86_64-elf-gcc -O0 -g -nostartfiles -L"$LIBDIR" -I"$INC_DIR" \
-o system_x86_64.elf system.c syscalls_x86_64.o -lc -lgcc
/home/kali/toolchains/x86_64-toolchain/bin/../lib/gcc/x86_64-elf/7.1.0/../../../../x86_64-elf/bin/ld: warning: cannot find entry symbol _start; defaulting to 0000000004000b0
/home/kali/toolchains/x86_64-toolchain/bin/../lib/gcc/x86_64-elf/7.1.0/../../../../x86_64-elf/bin/ld: warning: cannot find entry symbol _start; defaulting to 0000000004000b0
/home/kali/toolchains/x86_64-toolchain/bin/../lib/gcc/x86_64-elf/7.1.0/../../../../x86_64-elf/bin/ld: warning: cannot find entry symbol _start; defaulting to 0000000004000b0

(kali@kali) ~/lab1/newlib$
$ x86_64-elf-objdump -d -M intel stdio_x86_64.elf > stdio_x86_64.lst
x86_64-elf-objdump -d -M intel files_x86_64.elf > files_x86_64.lst
x86_64-elf-objdump -d -M intel system_x86_64.elf > system_x86_64.lst

```

-L"\$LIBDIR" і -I"\$INCDIR" потрібні, щоб компілятор і лінкер використовували саме newlib, яку я встановила, а не системну glibc. -L вказує, де брати бібліотеки (libc.a), інакше може не знайти; -I вказує, де брати заголовки (stdio.h тощо), інакше візьме невірні з /usr/include

Для stdio:

```
000000000400b0: <main>:
4000b0: 55                push    rbp
4000b1: 48 89 e5          mov     rbp,rsi
4000b4: ba a0 9f 40 00    mov     edx,0x409fa0
4000b9: be 2a 00 00 00    mov     esi,0x2a
4000be: bf a6 9f 40 00    mov     edi,0x409fa6
4000c3: b8 00 00 00 00    mov     eax,0x0
4000c8: e8 e3 01 00 00    call   4002b0 <printf>
4000cd: bf b5 9f 40 00    mov     edi,0x409fb5
4000d2: e8 39 03 00 00    call   400410 <puts>
4000d7: b8 00 00 00 00    mov     eax,0x0
4000dc: 5d                pop     rbp
4000dd: c3                ret
```

```
000000000400210: <_printf_r>:
400210: 48 81 ec d8 00 00 sub     rsp,0xd8
400217: 84 c0             test    al,al
400219: 48 89 54 24 30    mov     QWORD PTR [rsp+0x30],rdx
40021e: 48 89 4c 24 38    mov     QWORD PTR [rsp+0x38],rcx
400223: 4c 89 44 24 40    mov     QWORD PTR [rsp+0x40],r8
400228: 4c 89 4c 24 48    mov     QWORD PTR [rsp+0x48],r9
40022d: 74 37             je      400266 <_printf_r+0x56>
40022f: 0f 29 44 24 50    movaps  XMMWORD PTR [rsp+0x50],xmm0
400234: 0f 29 4c 24 60    movaps  XMMWORD PTR [rsp+0x60],xmm1
400239: 0f 29 54 24 70    movaps  XMMWORD PTR [rsp+0x70],xmm2
40023e: 0f 29 9c 24 80 00 movaps  XMMWORD PTR [rsp+0x80],xmm3
400245: 00
400246: 0f 29 a4 24 90 00 movaps  XMMWORD PTR [rsp+0x90],xmm4
40024d: 00
40024e: 0f 29 ac 24 a0 00 movaps  XMMWORD PTR [rsp+0xa0],xmm5
400255: 00
400256: 0f 29 b4 24 b0 00 movaps  XMMWORD PTR [rsp+0xb0],xmm6
40025d: 00
40025e: 0f 29 bc 24 c0 00 movaps  XMMWORD PTR [rsp+0xc0],xmm7
400265: 00
400266: 48 8d 84 24 e0 00 lea     rax,[rsp+0xe0]
40026d: 00
40026e: 48 89 f2          mov     rdx,rsi
400271: 48 8d 4c 24 08    lea     rcx,[rsp+0x8]
400276: 48 8b 77 10       mov     rsi,QWORD PTR [rdi+0x10]
40027a: 48 89 44 24 10    mov     QWORD PTR [rsp+0x10],rax
40027f: 48 8d 44 24 20    lea     rax,[rsp+0x20]
400284: c7 44 24 08 10 00 mov     DWORD PTR [rsp+0x8],0x10
40028b: 00
40028c: c7 44 24 0c 30 00 mov     DWORD PTR [rsp+0xc],0x30
400293: 00
400294: 48 89 44 24 18    mov     QWORD PTR [rsp+0x18],rax
400299: e8 12 02 00 00    call   4004b0 <_vfprintf_r>
40029e: 48 81 c4 d8 00 00 add     rsp,0xd8
4002a5: c3                ret
4002a6: 66 2e 0f 1f 84 00 nop     WORD PTR cs:[rax+rax*1+0x0]
4002ad: 00 00 00
```

```
000000000400360: <_puts_r>:
400360: 55                push    rbp
400361: 53                push    rbx
400362: 48 89 fb          mov     rbx,rdi
400365: 48 89 f7          mov     rdi,rsi
400368: 48 89 f5          mov     rbp,rsi
40036b: 48 83 ec 58       sub     rsp,0x58
40036f: e8 ac 00 00 00    call   400420 <strlen>
400374: 48 89 44 24 38    mov     QWORD PTR [rsp+0x38],rax
400379: 48 83 c0 01       add     rax,0x1
40037d: 48 83 7b 40 00    cmp     QWORD PTR [rbx+0x40],0x0
400382: 48 89 44 24 20    mov     QWORD PTR [rsp+0x20],rax
400387: 48 8d 44 24 30    lea     rax,[rsp+0x30]
40038c: 48 89 6c 24 30    mov     QWORD PTR [rsp+0x30],rbp
400391: 48 c7 44 24 40 c7 9f mov     QWORD PTR [rsp+0x40],0x409fc7
400398: 40 00
40039a: 48 c7 44 24 48 01 00 mov     QWORD PTR [rsp+0x48],0x1
4003a1: 00 00
4003a3: 48 89 44 24 10    mov     QWORD PTR [rsp+0x10],rax
4003a8: c7 44 24 18 02 00 mov     DWORD PTR [rsp+0x18],0x2
4003af: 00
4003b0: 48 8b 73 10       mov     rsi,QWORD PTR [rbx+0x10]
4003b4: 74 42 81/newlib   je      4003f8 <_puts_r+0x98>
4003b6: 0f b7 46 10       movzx   eax,WORD PTR [rsi+0x10]
```


У функції `_printf_r` видно, що аргументи передаються через регістри (`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`) і додатково зберігаються у стеку. Якщо є SIMD-аргументи, вони копіюються із `xmm0`–`xmm7` у стек. Далі готується структура з параметрами (через `lea`, `mov` у стек), і викликається внутрішня функція `_vfprintf_r`. У `_puts_r` спершу викликається `strlen`, після чого формується блок параметрів у стеку, перевіряються прапорці потоку, і викликається низькорівнева функція `__sfvwrite_r` для фактичного запису

Для Files:

```
000000000400b0: <main>:
4000b0: 55                push  rbp
4000b1: 48 89 e5          mov   rbp,rsi
4000b4: 48 83 ec 40       sub   rsp,0x40
4000b8: 48 c7 45 f8 7b 3b 40 mov   QWORD PTR [rbp-0x8],0x403b7b
4000bf: 00
4000c0: 48 8b 45 f8       mov   rax,QWORD PTR [rbp-0x8]
4000c4: be 84 3b 40 00    mov   esi,0x403b84
4000c9: 48 89 c7          mov   rdi,rax
4000cc: e8 0f 0b 00 00    call 400be0 <fopen>
4000d1: 48 89 45 f0       mov   QWORD PTR [rbp-0x10],rax
4000d5: 48 c7 45 e8 87 3b 40 mov   QWORD PTR [rbp-0x18],0x403b87
4000dc: 00
4000dd: 48 8b 45 e8       mov   rax,QWORD PTR [rbp-0x18]
4000e1: 48 89 c7          mov   rdi,rax
4000e4: e8 47 17 00 00    call 401830 <strlen>
4000e9: 48 89 c6          mov   rsi,rax
4000ec: 48 8b 55 f0       mov   rdx,QWORD PTR [rbp-0x10]
4000f0: 48 8b 45 e8       mov   rax,QWORD PTR [rbp-0x18]
4000f4: 48 89 d1          mov   rcx,rdx
4000f7: 48 89 f2          mov   rdx,rsi
4000fa: be 01 00 00 00    mov   esi,0x1
4000ff: 48 89 c7          mov   rdi,rax
400102: e8 b9 0e 00 00    call 400fc0 <fwrite>
400107: 48 8b 45 f0       mov   rax,QWORD PTR [rbp-0x10]
40010b: 48 89 c7          mov   rdi,rax
40010e: e8 9d 02 00 00    call 4003b0 <fclose>
400113: 48 8b 45 f8       mov   rax,QWORD PTR [rbp-0x8]
400117: be 8f 3b 40 00    mov   esi,0x403b8f
40011c: 48 89 c7          mov   rdi,rax
40011f: e8 bc 0a 00 00    call 400be0 <fopen>
400124: 48 89 45 f0       mov   QWORD PTR [rbp-0x10],rax
400128: 48 8b 55 f0       mov   rdx,QWORD PTR [rbp-0x10]
40012c: 48 8d 45 c0       lea   rax,[rbp-0x40]
400130: 48 89 d1          mov   rcx,rdx
```

```
000000000400b10: <_fopen_r>:
400b10: 41 55             push  r13
400b12: 41 54             push  r12
400b14: 49 89 f5          mov   r13,rsi
400b17: 55              push  rbp
400b18: 53              push  rbx
400b19: 48 89 d6          mov   rsi,rdx
400b1c: 48 89 fd          mov   rbp,rdi
400b1f: 48 83 ec 18       sub   rsp,0x18
400b23: 48 8d 54 24 0c    lea   rdx,[rsp+0xc]
400b28: e8 03 20 00 00    call 402b30 <__sflags>
400b2d: 85 c0             test  eax,eax
400b2f: 74 7f            je    400bb0 <_fopen_r+0xa0>
400b31: 48 89 ef          mov   rdi,rbp
400b34: 41 89 c4          mov   r12d,eax
400b37: e8 f4 fd ff ff    call 400930 <__sfp>
400b3c: 48 85 c0          test  rax,rax
400b3f: 48 89 c3          mov   rbx,rax
400b42: 74 6c            je    400bb0 <_fopen_r+0xa0>
400b44: 8b 54 24 0c       mov   edx,DWORD PTR [rsp+0xc]
400b48: b9 b6 01 00 00    mov   ecx,0x1b6
400b4d: 4c 89 ee          mov   rsi,r13
400b50: 48 89 ef          mov   rdi,rbp
400b53: e8 48 0f 00 00    call 401aa0 <_open_r>
400b58: 85 c0             test  eax,eax
400b5a: 78 44            js    400ba0 <_fopen_r+0x90>
400b5c: 66 44 89 63 10    mov   WORD PTR [rbx+0x10],r12w
400b61: 41 81 e4 00 01 00 00 and   r12d,0x100
400b68: 66 89 43 12       mov   WORD PTR [rbx+0x12],ax
400b6c: 48 89 5b 38       mov   QWORD PTR [rbx+0x38],rbx
400b70: 48 c7 43 40 90 12 40 mov   QWORD PTR [rbx+0x40],0x401290
400b77: 00
400b78: 48 c7 43 48 d0 12 40 mov   QWORD PTR [rbx+0x48],0x4012d0
400b7f: 00
400b80: 48 c7 43 50 30 13 40 mov   QWORD PTR [rbx+0x50],0x401330
400b87: 00
```

У `main` спочатку через `mov` і `call` викликається `fopen` з іменем файлу і режимом. Потім аргументи готуються для `strlen` і `fwrite`, які записують дані у файл, після чого виконується `fclose`. Далі файл відкривається знову, виконується виклик `fread` з буфером, і в кінці буфер завершується нуль-байтом.

Усередині `_foren_r` видно, що режим перевіряється через `__sflags`, виділяється структура `FILE` через `__sfr`, після чого виконується `_open_r`. Якщо відкриття успішне — структура ініціалізується показниками на функції обробки.

Для System:

```
00000000004000b0 <main>:
4000b0: 55                push    rbp
4000b1: 48 89 e5          mov     rbp, rsp
4000b4: 48 83 ec 10       sub     rsp, 0x10
4000b8: bf 40 9f 40 00    mov     edi, 0x409f40
4000bd: e8 7e 01 00 00    call   400240 <system>
4000c2: 89 45 fc          mov     DWORD PTR [rbp-0x4], eax
4000c5: 8b 45 fc          mov     eax, DWORD PTR [rbp-0x4]
4000c8: 89 c6             mov     esi, eax
4000ca: bf 57 9f 40 00    mov     edi, 0x409f57
4000cf: b8 00 00 00 00    mov     eax, 0x0
4000d4: e8 37 02 00 00    call   400310 <printf>
4000d9: b8 00 00 00 00    mov     eax, 0x0
4000de: c9               leave   eax
4000df: c3               ret
```

```
0000000000400210 <_system_r>:
400210: 48 85 f6          test    rsi, rsi
400213: 75 0b             jne     400220 <_system_r+0x10>
400215: 31 c0             xor     eax, eax
400217: c3               ret
400218: 0f 1f 84 00 00 00 00 nop     DWORD PTR [rax+rax*1+0x0]
40021f: 00
400220: 48 83 ec 08       sub     rsp, 0x8
400224: e8 97 01 00 00    call   4003c0 <__errno>
400229: c7 00 58 00 00 00 mov     DWORD PTR [rax], 0x58
40022f: b8 ff ff ff ff    mov     eax, 0xffffffff
400234: 48 83 c4 08       add     rsp, 0x8
400238: c3               ret
400239: 0f 1f 80 00 00 00 00 nop     DWORD PTR [rax+0x0]
```

У `main` у регістр `edi` завантажується адреса рядка-команди і викликається `system`. Результат (код завершення) зберігається у стек, потім підставляється у `esi` і виводиться через `printf`. У `_system_r` видно просту перевірку: якщо аргумент `NULL`, повертається 0; інакше викликається `__errno`, записується помилка `ENOSYS` (0x58) та повертається -1. Тобто тут виклик `system` фактично не виконує команд, а лише імітує їх.